Assignment No 8
No SQL : Aggregation & Indexing
Design & Develop queries using Aggregation & Indexing

Indexing:-

1. Problem Statement:
Create an index on Salary to improve search performance.

```
EmployeeAppraisalSystem> db.Employee.createIndex({ Salary: 1 })
Salary_1
```

2. Problem Statement:
Find employees having Salary greater than 60000 (uses index).

```
EmployeeAppraisalSystem> db.Employee.find({ Salary: { $gt: 60000 } })
[
  {
    _id: ObjectId('701a1'),
    Emp_ID: 102,
    Emp_Name: 'Prachi',
    Designation: 'Developer',
    Salary: 70000,
    Dept_ID: 2
  }
]
```

3. Problem Statement:
Create an index on Salary without blocking database operations.

```
EmployeeAppraisalSystem> db.Employee.createIndex(
  { Salary: 1 },
  { background: true }
```

```
)
Salary_1
```

4. Problem Statement:
Create an index with a custom name on employee salary.

```
EmployeeAppraisalSystem> db.Employee.createIndex(
  { Salary: 1 },
  { name: "Salary_Index" }
)
Salary_Index
```

5. Problem Statement:
Create index only for documents that contain Designation field.

```
EmployeeAppraisalSystem> db.Employee.createIndex(
  { Designation: 1 },
  { sparse: true }
)
Designation_1
```

6. Problem Statement:
Automatically delete appraisal records after a certain time (example: 1 year).

```
EmployeeAppraisalSystem> db.Appraisal.createIndex(
  { appraisalDate: 1 },
  { expireAfterSeconds: 31536000 }
)
appraisalDate_1
```

Aggregation:-

1. Problem Statement:
Find the number of employees having Salary greater than 60000.

```
EmployeeAppraisalSystem> db.Employee.aggregate([
  { $match: { Salary: { $gt: 60000 } } },
  { $count: "High_Salary_Employees" }
])
[
  { High_Salary_Employees: 1 }
]
```

2. Problem Statement:
Find the average salary of employees based on their designation.

```
EmployeeAppraisalSystem> db.Employee.aggregate([
  {
   $group: {
     _id: "$Designation",
     Avg_Salary: { $avg: "$Salary" }
   }
  }
])
[
  { _id: 'Developer', Avg_Salary: 65000 },
  { _id: 'Senior Manager', Avg_Salary: 60000 },
  { _id: 'Analyst', Avg_Salary: 50000 },
  { _id: 'AIML', Avg_Salary: 50000 }
]
```

3. Problem Statement:
Find the number of employees working in each department.

```
EmployeeAppraisalSystem> db.Employee.aggregate([
  {
    $group: {
      _id: "$Dept_ID",
      Total_Employees: { $sum: 1 }
    }
  }
])
[
  { _id: 1, Total_Employees: 1 },
  { _id: 2, Total_Employees: 3 },
  { _id: 3, Total_Employees: 1 }
]
```

4. Problem Statement:
Find the minimum performance score among employees.

```
EmployeeAppraisalSystem> db.Appraisal.aggregate([
  {
    $group: {
      _id: null,
      Min_Score: { $min: "$performance_score" }
    }
  }
])
[
  { _id: null, Min_Score: 5 }
]
```

5. Problem Statement:
Find the maximum and minimum salary of employees in each department.

```
EmployeeAppraisalSystem> db.Employee.aggregate([
  {
   $group: {
     _id: "$Dept_ID",
     Max_Salary: { $max: "$Salary" },
     Min_Salary: { $min: "$Salary" }
   }
  }
])
[
  { _id: 1, Max_Salary: 60000, Min_Salary: 60000 },
  { _id: 2, Max_Salary: 70000, Min_Salary: 50000 },
  { _id: 3, Max_Salary: 50000, Min_Salary: 50000 }
]
```

6. Problem Statement:
Find average performance score based on rating.

```
EmployeeAppraisalSystem> db.Appraisal.aggregate([
  {
   $group: {
     _id: "$rating",
     Avg_Score: { $avg: "$performance_score" }
   }
  }
])
[
  { _id: 'Excellent', Avg_Score: 9 },
  { _id: 'Good', Avg_Score: 7 },
  { _id: 'Average', Avg_Score: 5 }
]
```